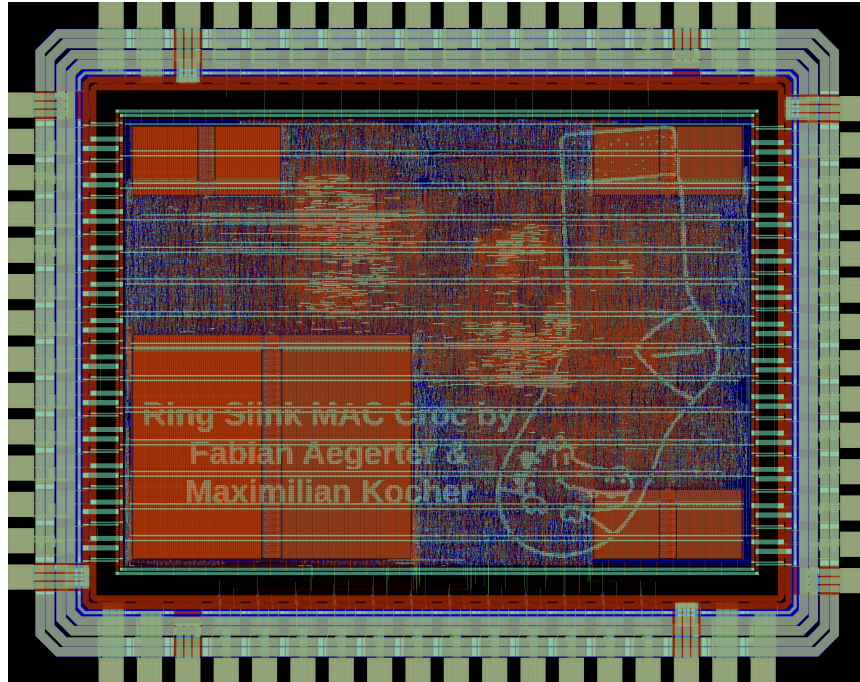




Ringbus for Inference Acceleration

Distributing matrix-vector inference across multiple Croc instances over a uni-directional ring bus

Fabian Aegerter
Maximilian Kocher

Project Report for VLSI 2

Spring Term 2026

Integrated Systems Laboratory

ETH

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

1 Introduction

As part of a semester thesis, *Fabian Aegerter* and *Llorenç Muela Hausmann* developed a unidirectional ring bus extension for **Croc**, a lightweight System-on-Chip designed for educational purposes built on PULP IPs. This extension enables multiple Croc instances to be coupled computationally, allowing them to collaborate on complex problems. Given the ring bus’s lower bandwidth and higher latency relative to the internal OBI bus, the architecture is best suited for workloads that are computationally intensive but have low communication overhead.

A workload well suited for the ring bus is machine learning inference, where the output y depends on the multiplication of the weight matrix $\mathbf{W}$ and an input vector x : $y = \mathbf{W} \cdot x$. When the problem is sufficiently large, i.e. the weight matrix $\mathbf{W} \in \mathbb{Q}^{R \times C}$ is large, then the computation becomes the dominant bottleneck, making the ring bus’s limited bandwidth less of a constraint. The workload is parallelized by distributing the rows of $\mathbf{W}$ across multiple Croc instances, each of which computes a disjoint partition of the output vector y .

Distributing $\mathbf{W}$ across multiple Croc instances theoretically enables inference on larger weight matrices, as the memory requirement is spread across the local SRAMs of all instances rather than being constrained to a single one.

In addition to the multi-Croc speedup, a single Croc is accelerated for matrix vector multiplication by a dedicated hardware accelerator.

This project aims to demonstrate a performance speedup in matrix-vector multiplication over a single Croc instance utilizing the ALU with a dedicated HW multiplier (which was not used in the standard baseline of Croc). Additionally, it aims to demonstrate how multiple Crocs in a ring bus topology can be configured to solve tasks with high data and compute requirements.

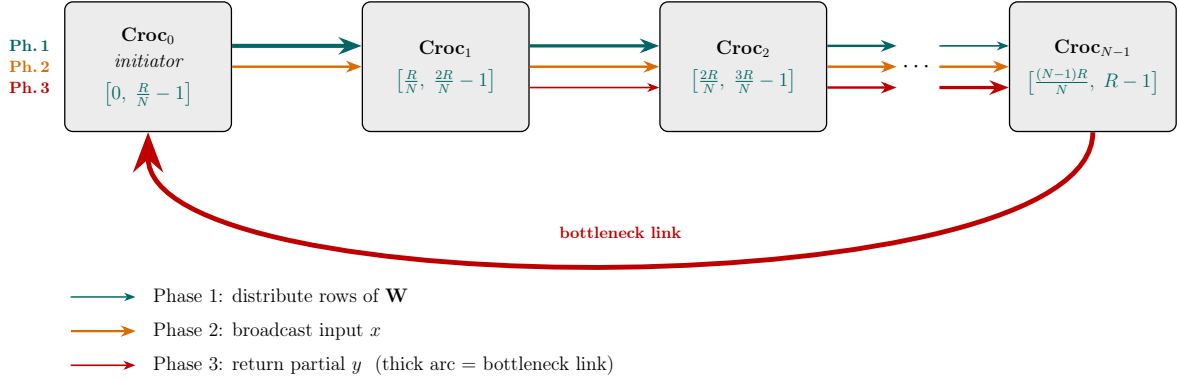


Figure 1: Unidirectional ring bus with N Croc instances. Arrow thickness reflects communication load. **Phase 1**: Croc₀ pre-distributes disjoint row-blocks of $\mathbf{W}$ to each node. **Phase 2**: input vector x is broadcasted. **Phase 3**: partial results $y_i = \mathbf{W}_i x$ are forwarded hop-by-hop toward Croc₀ (initiator); the return link from Croc_{N-1} to Croc₀ carries the fully accumulated result (bottleneck).

2 Calculations

To justify the project, an analytical model is first derived to estimate the expected speedup gained by distributing the inference workload across multiple Croc instances, relative to a single-Croc baseline. The following variables are defined for this purpose:

- N_{Croc} = Number of Croc instances ≥ 1
- $\mathbf{W} = \mathbf{W} \in \mathbb{Q}^{R \times C}$ - weight matrix with R rows, C columns
- M = constant $R \cdot C$ product
- Q_{ring} = Ring bus throughput [words / cycle]
- Q_{obi} = Croc OBI bus throughput [words / cycle]

- L = Per-hop latency [cycles] between adjacent Croc instances
- K_{corr} = Correction factor accounting for roughly the number of instructions required per multiply-accumulate (MAC) operation. E.g., loading two operands from SRAM (2 instructions), performing a multiplication and accumulation (2 instructions) (assuming a fast single cycle HW multiplier). Writing the result back to SRAM is not modeled, since the accumulated value is stored in the register file of the CPU. A total of $K_{corr} = 4$ instructions per MAC operation are required.

2.1 Cost Function

A cost function, expressed in cycles, is derived by the sum of contributions for each phase of the distributed inference execution - such as the collection of the partial output vectors y from each Croc instance. This cost function is the foundation for the analysis presented in the following sections.

2.1.1 Phase 1: $\mathbf{W}$ distribution

The weight matrix is distributed across all Croc instances, with each instance receiving and storing $\frac{R \cdot C}{N}$ words in its local SRAM. Croc instance i completes its reception at time $i \cdot L + \frac{i \cdot R \cdot C}{N \cdot Q_{ring}}$. The first phase concludes when the final instance in the ring chain has received its partition of $\mathbf{W}$, which occurs after T_1 cycles:

$$T_1 = (N - 1) \cdot L + \frac{(N - 1) \cdot R \cdot C}{N \cdot Q_{ring}} \quad (1)$$

2.1.2 Phase 2: Propagation of x

The vector x with its C words is distributed across all Croc instances. Croc instance i completes its reception at time $i \cdot L + \frac{C}{Q_{ring}}$. This second phase concludes when the final instance in the ring chain has received vector x , which occurs at T_2 cycles:

$$T_2 = \begin{cases} (N - 1) \cdot L + \frac{C}{Q_{ring}} & \text{if } N > 1 \\ 0 & \text{if } N = 1 \end{cases} \quad (2)$$

2.1.3 Phase 3: Computation and Propagation of y

If the system isn't properly tuned, then it is either bandwidth or computationally limited. If the system is bandwidth limited, then it means that the vector y can't be returned fast enough from all Croc's back to the initiating Croc of the computation, compared to how fast the matrix multiplication gets executed.

The bottleneck of the ring bus is the link from Croc $_{N-1}$ to Croc $_0$ (the initiating Croc of the computation), as every element of the output vector y must traverse it. In the optimal case, the time required to transmit the accumulated partial results of all $N - 1$ instances across this link exactly matches the time each instance needs to compute a single dot product between one row of $\mathbf{W}$ and the input vector x , producing one element of y . This balance ensures that no instance is idle, and the pipeline is fully utilised. This can be summarized as $\max(\frac{C \cdot K_{corr} + 1}{Q_{obi}}; \frac{N - 1}{Q_{ring}})$.

The compute term $\frac{C \cdot K_{corr} + 1}{Q_{obi}}$ models the cost of processing a single row of $\mathbf{W}$. For each row, C elements of $\mathbf{W}$ and C elements of x are loaded from SRAM, followed by $2 \cdot C$ arithmetic operations (each MAC consists of one multiplication and one addition), and finally one store of the resulting scalar. This yields $K_{corr} = 4$ cycles per element (assuming $Q_{obi} = 1$ and a fast HW multiplier), giving a total of $\frac{C \cdot K_{corr} + 1}{Q_{obi}}$ cycles per row. Note that this model theoretically assumes x is always reloaded from SRAM for every row; in practice, if $C \lesssim 16$, the vector x may fit entirely in the CPU register file, eliminating these redundant loads.

Noting that Croc $_1$ begins computation earlier having received the input vector x first, it can be assumed that Croc $_1$ partial result arrives at Croc $_{N-1}$ precisely when Croc $_{N-1}$ completes its own computation.

Combining these observations, the system achieves perfect balance between compute and communication when their per-row processing rates are equal:

$$\frac{C \cdot K_{corr} + 1}{Q_{obi}} = \frac{N - 1}{Q_{ring}} \quad (3)$$

However, in the compute-bound regime, the total execution time must also account for the final pipeline flush. After all $\frac{R}{N}$ rows of W have been processed, an additional transmission delay $\frac{N-1}{Q_{ring}}$ is incurred to push the final result vector y onto the ring bus. Therefore, the total execution time T_3 is given by:

$$T_3 = \max \left(\frac{R}{N} \cdot \frac{C \cdot K_{corr} + 1}{Q_{obi}} + \frac{N - 1}{Q_{ring}}, \frac{R}{N} \cdot \frac{N - 1}{Q_{ring}} \right) + \begin{cases} L & \text{if } N > 1 \\ 0 & \text{if } N = 1 \end{cases} \quad (4)$$

2.1.4 Total cost Function

Summing the costs of all three phases from Sections 2.1.1, 2.1.2, and 2.1.3 yields the total cost function in cycles

$$\begin{aligned} T_{total} &= T_1 + T_2 + T_3 \\ &= \underbrace{(N - 1)L + \frac{(N - 1)RC}{NQ_{ring}}}_{\text{Phase 1}} + \underbrace{\begin{cases} (N - 1)L + \frac{C}{Q_{ring}} & \text{if } N > 1 \\ 0 & \text{if } N = 1 \end{cases}}_{\text{Phase 2}} \\ &\quad + \underbrace{\max \left(\frac{R}{N} \frac{CK_{corr} + 1}{Q_{obi}} + \frac{N - 1}{Q_{ring}}, \frac{R}{N} \frac{N - 1}{Q_{ring}} \right) + \begin{cases} L & \text{if } N > 1 \\ 0 & \text{if } N = 1 \end{cases}}_{\text{Phase 3}} \end{aligned} \quad (5)$$

Dividing the cost function of the single Croc by the multi Croc system yields the expected speedup, as plotted in Figure 2.

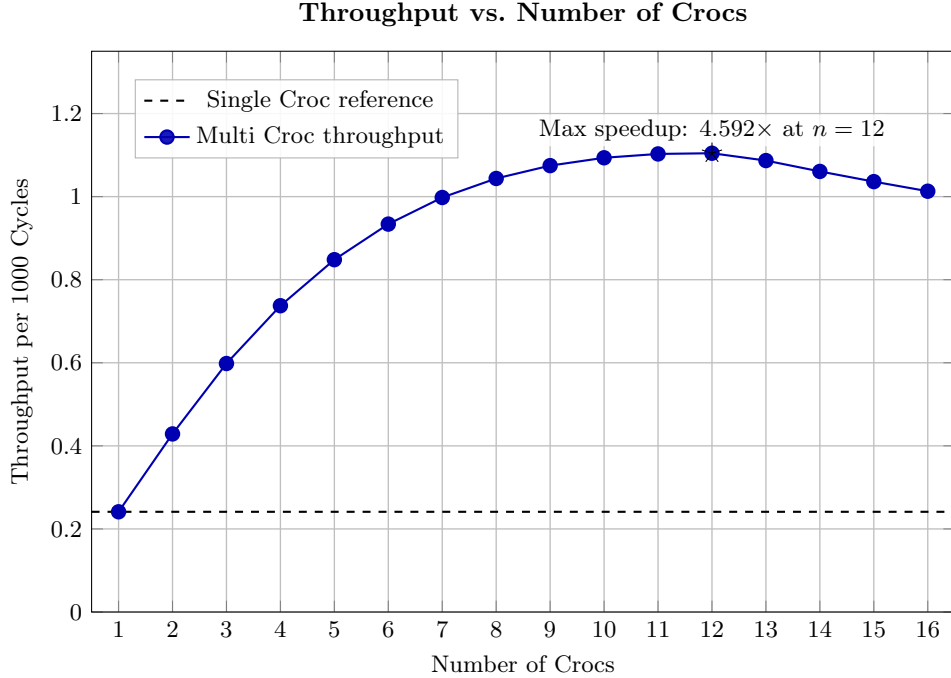


Figure 2: Throughput vs. number of Crocs. Parameters: $L = 20$, $Q_{ring} = 0.1$ (numbers/cycle), $Q_{obi} = 1$ (per cycle), $R = 49$, $C = 21$, $k = 4$.

2.2 Comparison of Speedup to a Single Croc

Putting $N = 1$ into equation (5) yields $R \cdot \frac{C \cdot K_{corr} + 1}{Q_{obi}}$, corresponding to the number of cycles one Croc needs to compute the matrix vector multiplication. Furthermore, for large N and $R \cdot C$ equation (5) can be approximated as $2NL + \frac{RC}{Q_{ring}} + \frac{RC K_{corr}}{N Q_{obi}} + \frac{N}{Q_{ring}}$.

Minimizing over N by taking the derivative and setting it to 0 yields the optimal number of Croc instances:

$$N^* = \sqrt{\frac{R \cdot C \cdot K_{corr}}{Q_{obi} \left(2L + \frac{1}{Q_{ring}}\right)}} \quad (6)$$

Substituting back, the minimum achievable cost is:

$$T_{approx}(N^*) = 2\sqrt{\frac{RC K_{corr}}{Q_{obi} \left(2L + \frac{1}{Q_{ring}}\right)}} + \frac{RC}{Q_{ring}} \quad (7)$$

For the distributed system to outperform the single-Croc baseline, $T_{approx}(N^*) < \frac{RC K_{corr}}{Q_{obi}}$ must hold, where for the single Croc instance it was again assumed that RC is large:

$$2\sqrt{\frac{RC K_{corr}}{Q_{obi} \left(2L + \frac{1}{Q_{ring}}\right)}} + \frac{RC}{Q_{ring}} < \frac{RC K_{corr}}{Q_{obi}} \quad (8)$$

$$2\sqrt{\frac{K_{corr}}{RC Q_{obi} \left(2L + \frac{1}{Q_{ring}}\right)}} + \frac{1}{Q_{ring}} < \frac{K_{corr}}{Q_{obi}} \quad (9)$$

$$\frac{K_{corr}}{Q_{obi}} - \frac{1}{Q_{ring}} > 2\sqrt{\frac{K_{corr}}{RC Q_{obi} \left(2L + \frac{1}{Q_{ring}}\right)}} \quad (10)$$

Since the right-hand side is strictly positive, this condition requires $\frac{K_{corr}}{Q_{obi}} - \frac{1}{Q_{ring}} > 0$, i.e. $Q_{ring} > \frac{Q_{obi}}{K_{corr}}$. For the current hardware parameters ($K_{corr} = 4$, $Q_{obi} = 1$, $Q_{ring} = 0.2$), the left-hand side evaluates to $4 - 5 = -1 < 0$, so the inequality cannot be satisfied for any matrix dimension. Consequently, either the ring bus bandwidth must satisfy $Q_{ring} > \frac{Q_{obi}}{K_{corr}} = 0.25$ words/cycle, or $\mathbf{W}$ must be pre-distributed across all Croc instances. The latter is consistent with established practice in LLM inference where weight matrices are pre-distributed across compute nodes. Furthermore, the distribution overhead can be amortized by performing sufficiently many matrix-vector multiplications with the same $\mathbf{W}$, reducing the per-inference cost of Phase 1 towards zero.

3 Single Croc Optimization

Despite the distributed system optimizations, a significant portion of a single Croc chip’s area remains unutilized. Scaling inference across multiple Crocs is difficult to justify without first maximizing intra-Croc performance. This section evaluates architectural optimizations to minimize the cycles per operation (K_{corr}).

A straight forward approach to accelerate computation is leveraging Single Instruction, Multiple Data (SIMD) paradigms. In a 32-bit architecture, quantizing data to 8 bits allows four operands to be fetched and processed simultaneously, theoretically yielding a $4\times$ increase in throughput. However, this gain comes at the direct cost of numerical precision. Although 8-bit quantization provides sufficient accuracy for most machine learning applications, relying on data reduction obscures true hardware efficiency. To ensure a fair baseline and isolate purely architectural speedups, all evaluations in this study remain strictly constrained to 32-bit numbers.

An approach to reduce k is by implementing a pipelined MAC accelerator, which fetches one value from the weightmatrix each cycle. Storing both the weight matrix (W) and input vector (X) in the same SRAM (bank) forces the interconnect to serialize access. Partitioning W and X into separate SRAM banks allows for parallel data fetching, eliminating this memory contention. This concept can be further pushed by distributing W across multiple SRAM banks, allowing the multiple OBI managers of the accelerator to fetch multiple weight values every cycle. If the input vector (X) is stored locally in

the MAC accelerator in D-flip-flop registers (roughly 100 elements, with 32 bits each), the bus is freed to stream exclusively W . Processing this parallel feed through multiple MAC accelerators achieves ($k < 1$), moving the ultimate performance constraint from the interconnect bandwidth to the physical chip area (mm^2).

For this project, a single MAC accelerator was implemented with a dedicated SRAM bank for the input vector $\mathbf{x}$. By storing $\mathbf{x}$ in SRAM rather than a dedicated register file, the MAC accelerator can fetch both a weight matrix element $W[i]$ and the corresponding $x[i]$ in the same cycle via two parallel OBI manager ports. This eliminates the need for D flip-flops in a vector register file, resulting in a significant area saving, as noticed by trial and error.

4 Design Additions

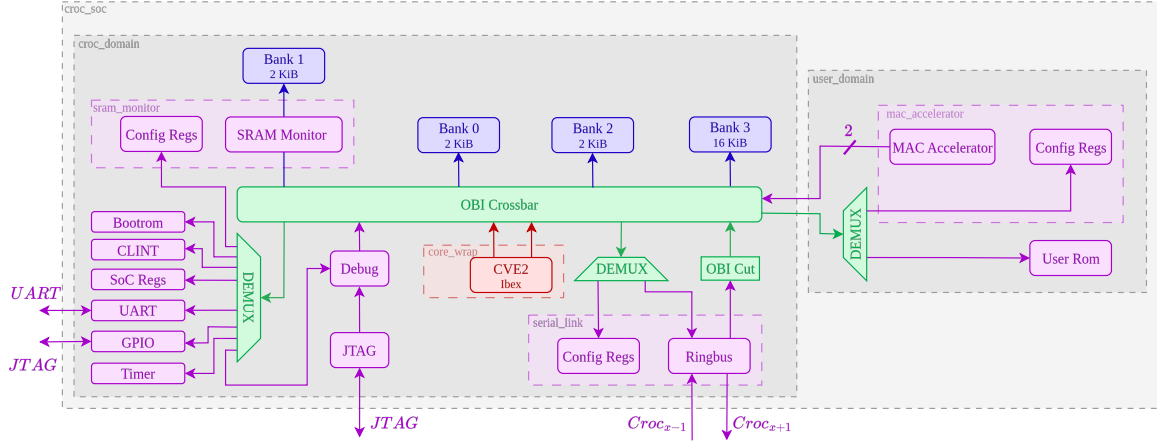


Figure 3: Block diagram of the Croc SoC with the design additions

4.1 MAC Accelerator

The MAC accelerator consists of two building blocks: the pipelined MAC core and a control register file.

4.1.1 MAC Accelerator Core (mac_accelerator.sv)

The MAC accelerator computes the matrix-vector product $y = \mathbf{W}x$ row by row, with both $\mathbf{W}$ and $\mathbf{x}$ stored in SRAM in separate banks. It implements a 3-stage pipeline: Stage 0 issues two parallel OBI read requests to fetch $W[i]$ and $x[i]$ simultaneously from their respective SRAM banks, Stage 1 multiplies the fetched elements, and Stage 2 accumulates the products into a running sum.

The FSM has four states: IDLE, FETCH, DRAIN, and DONE. Computation begins on a single-cycle `start_i` pulse triggered by a CPU write to the control register file. After the last fetch, the FSM enters DRAIN for two cycles to flush the remaining pipeline stages, then asserts an interrupt in the DONE state and advances automatically to the next row until all `num_row` rows are computed. The FSM holds in DONE until the CPU clears the interrupt, ensuring the result is not overwritten before it has been forwarded to the initiating Croc ($Croc_0$). This creates implicit flow control: if sending the result back to the initiating Croc takes longer than computing the next row, the MAC stalls; if computation takes longer, the send waits.

Cycle	-1	0	1	2	...	$N-1$	N	$N+1$	$N+2$
Stage 0 (fetch)	—	$W[0]$	$W[1]$	$W[2]$	...	$W[N-1]$	—	—	—
Stage 1 (mult.)	—	—	$W[0]$	$W[1]$	...	$W[N-2]$	$W[N-1]$	—	—
Stage 2 (accum.)	—	—	—	$p[0]$	...	$p[N-3]$	$p[N-2]$	$p[N-1]$	—
FSM state	IDLE	FETCH					DRAIN		DONE

Table 1: MAC accelerator 3-stage pipeline timing (N elements). *Notes:* $p[i] = W[i] \cdot x[i]$ denotes the partial product from cycle i .

4.1.2 Control Register File (mac_ctrl_regs.sv)

The control register file provides the CPU-facing configuration and status interface over OBI. Its register map is in Table 2.

Register	Offset	Access	Reset	Description
matrix_base_addr	0x00	rw	0x0	Base address of the weight matrix $\mathbf{W}$ in SRAM.
vec_len	0x04	rw	0x0	Number of elements per row (vector length, $\leq \text{VecLen}$).
num_row	0x08	rw	0x0	Number of rows the MAC computes sequentially.
result	0x0C	r	—	Dot-product result of the last completed row (hardware-written).
status	0x10	r	0x0	done [0] - Done flag, latched on MAC completion and cleared on CPU read.
x_base_addr	0x1C	rw	0x0	Base address of the input vector $\mathbf{x}$ in SRAM.
start	0x24	rw	0x0	Writing any value triggers a single-cycle start pulse to the MAC and latches irq_start , which asserts IRQ line 22 on remote nodes; reading this register clears the latch.

Table 2: Configuration register map of the MAC accelerator module.

Two interrupts are generated: one mirrors the done flag and fires after each completed row, while the other fires when $y_counter \geq N_CROCS - 1$, signalling that all remote nodes have returned their results. The latter is suppressed for $N_CROCS = 1$, as there are no remote Crocs to return a result.

4.1.3 Interrupt Routing

Two interrupts are generated: **irq_o** mirrors the done flag and fires after each completed row and **irq_start_o** is latched when any node writes to the start register (offset 0x24) and cleared when the register is read back, used by remote nodes to synchronize computation start.

4.2 Ring Serial Link

The ring Serial Link IP acts as an interface for OBI transactions between multiple SoCs within a unidirectional ring bus topology. It uses the four most significant bits of the address to determine the target node within the ring and implements the routing such that source and destination are always served the signals of the OBI address and response channels. The details and specifications can be found in the report on it [1].

4.2.1 Ring Serial Link Modifications

The Ring Serial Link IP [1] has been modified to introduce a new payload type: the global write payload [2]. With this new payload type, the ring Serial Link IP no longer serves solely as an interface for OBI transactions; it additionally provides the functionality of writing to every node in the ring, independent of the number of nodes and without requiring a write response. The standard configuration has further been adapted to contain 10 lanes instead of 8, which allows 10 bits to be transmitted per cycle instead of 8. The packing of the payloads has also been adapted so that the four MSBs are no longer included in the body of the payload, since they are already stored in the header. This results in a 20% speedup for global write and write request payloads: removing four bits lets the payload be transmitted in one cycle fewer than before, reducing the transmission from 5 cycles to 4.

The global payloads enable a primitive address mapping, since the four MSBs denote whether a payload is a global payload or not. The mapping can also be deactivated by setting this value to 0, which is configured in the configuration registers of the ring Serial Link module. The resulting changes to the address mapping are listed in Table 3.

Register	Offset	Access	Reset	Description
GLOBAL_ADDR*	0x0C	rw	0xF	Holds the address value used to identify global write payloads via the four MSBs. Setting the register to 0x0 disables global addressing.
RAW_MODE_EN†	0x10	w	0x0	Enables raw mode, forwarding incoming data to the output for testing purposes.
RAW_MODE_IN_DATA†	0x14	r	—	Data received on the selected channel in raw mode (hardware-written).
RAW_MODE_IN_CH_SEL†	0x18	w	0x0	Selects the channel to read received data from in raw mode.
RAW_MODE_OUT_DATA_FIFO†	0x1C	w	0x0	Data pushed into the raw mode output FIFO.
RAW_MODE_OUT_DATA_FIFO_CTRL†	0x20	w	—	clear — Clears the raw mode TX FIFO.
		r	0x0	fill_state [15:8] — Number of entries currently held in the raw mode TX FIFO.
		r	0x0	is_full [31] — Set when the FIFO is full; further writes are ignored until space becomes available.
RAW_MODE_OUT_EN†	0x24	rw	0x0	Enables transmission of the data currently held in the output FIFO.
FLOW_CONTROL_FIFO_CLEAR†	0x28	w	0x0	Clears the flow-control FIFO, resetting the credit-based flow-control state.

Table 3: Changes to the configuration register map of the Ring Serial Link module.

* Newly added register.

† Existing register relocated by one word (0x04) due to the insertion of GLOBAL_ADDR.

4.3 SRAM Monitor

The SRAM Monitor module tracks accesses to an SRAM bank. It counts accesses to a configurable address range and, once a set number of accesses is reached, raises a CPU interrupt. To clear the interrupt and restart monitoring, software must then access the monitor’s control registers. In this design, the SRAM Monitor watches the second 512 word SRAM bank so that an interrupt is triggered once the transmission of the x vector has finished.

The monitor is fully configurable - the access type, the address range, and the access count are all set through its configuration registers, described below in Table 4

Register	Offset	Access	Reset	Description
CTRL	0x00	rw	0x0	enable [0] — Enables the SRAM monitor.
				clear [1] — Write-1 self-clearing pulse that resets COUNT and the pending IRQ while keeping enable unchanged.
THRESHOLD	0x04	rw	0x0	Access count after which the interrupt is generated. A value of 0 disables interrupt generation.
COUNT	0x08	r	0x0	Number of accesses within the monitored address range since the last clear or interrupt.
STATUS	0x0C	r	0x0	irq_pending [0] — Indicates that an interrupt is pending. Reading this register clears the pending interrupt.
START_ADDR	0x10	rw	0x0	Lowest tracked byte address within the SRAM bank (inclusive).
END_ADDR	0x14	rw	0x7FF	Highest tracked byte address within the SRAM bank (inclusive). The reset value covers the entire SRAM bank.
ACCESS	0x18	rw	0x0	Access type to monitor: 0 = write, 1 = read, 2 = read and write.

Table 4: SRAM Monitor configuration register map.

5 Test Setup

To evaluate the performance gains from the modifications to Croc, the speedup is benchmarked both with and without the accelerator for varying numbers of Croc instances. All tests were conducted with

the following parameters:

- System clock speed: 80 MHz
- Ring Serial Link:
 - Number of channels: 1
 - Number of lanes: 10
 - DDR: enabled
 - Clock division: 4
 - RX CDC FIFO depth: 4

5.1 Matrix Size Considerations

A key contribution of the ring-bus architecture is the ability to distribute a large weight matrix across multiple Croc chips. Since the matrix-vector multiplication scales as $O(N^2)$ while inter-Croc communication scales as $O(N)$, larger matrices yield disproportionately greater speedups. A direct comparison against a single baseline Croc with equivalent total memory is not feasible due to die area constraints. Three scenarios are therefore evaluated:

1. **Small matrix (fits in baseline):** The weight matrix occupies one 2 kB SRAM bank, allowing a fair comparison against a single unmodified Croc.
2. **Area-filling matrix:** The weight matrix is sized to fill the largest available SRAM of a single Croc (≈ 16 kB), representing the maximum single-chip capacity.
3. **Distributed matrix (proof of concept):** The full ring-bus system of 14 Crocs is utilized, each contributing 8 kB of SRAM, yielding a combined weight matrix of 14×16 kB = 224 kB. No single-Croc baseline exists at this scale.

5.2 Timing Methodology

In simulation, the JTAG controller initializes each Croc sequentially, introducing startup skew of up to $150 \mu\text{s}$ between the first and last node. For a single computation run finishing in approximately $300 \mu\text{s}$, this skew is significant. One approach to mitigate this is, each test repeats the computation 100 times after JTAG initialization, reducing the JTAG contribution to roughly 1% of total runtime. Using the first node's resume time as the start reference yields the most conservative speedup estimate. In hardware deployment, all Crocs would reset and start synchronously, eliminating this issue entirely.

A better approach is to use a cycle-accurate measurement of execution time using the RISC-V `mcycle` hardware counter, which increments by one every clock cycle. Start and end cycle counts, along with their difference, are written to a dedicated region in SRAM (`TIMING_ADDRESS`):

```
1 volatile uint32_t *timing = (volatile uint32_t *)TIMING_ADDRESS;
2 timing[0] = (uint32_t)get_mcycle(); // start
3 // ... code under measurement ...
4 timing[1] = (uint32_t)get_mcycle(); // end
5 timing[2] = timing[1] - timing[0]; // difference
```

Listing 1: Cycle-accurate timing measurement using the `mcycle` CSR.

After execution completes, the testbench reads these three values from SRAM via the JTAG debug interface and prints them to the terminal, yielding the cycle count of the measured region.

Additionally, Verifying the y-values inside the `N_TESTS` loop also adds overhead, further under-estimating the true speedup.

5.3 C program

The test program is compiled separately for each node with node-specific preprocessor defines (`NODE_ID`, `NUM_NODES`, `N_TESTS`), allowing a single source file to serve all nodes in the ring. The matrix dimensions `VEC_LEN` and `NUM_ROWS` are compile-time constants, with `NUM_ROWS` defined as a multiple of `NUM_NODES` to ensure each node owns an equal number of rows (`ROWS_PER_NODE`). The product of rows and columns is kept constant to follow the constraints made to the **W** as previously discussed. Whether the MAC accelerator is used is controlled by the `USE_MAC_ACCEL` preprocessor flag.

At startup, each node initializes its own portion of the weight matrix **W** in SRAM (all weights set to 1) and the input vector **x** (with $x[i] = i + 1$). The expected result is therefore $y = \sum_{i=1}^{\text{VEC_LEN}} i$, which is used for verification. It is important to note that initializing **W** and **x** with zero values artificially accelerates the software baseline, since in the standard Croc no hardware multiplier is instantiated and the software emulates the multiplication. When the values are 0, the multiplication is much faster compared to when the weightmatrix and the X vector are properly instantiated. The MAC accelerator always issues the same number of SRAM fetches and pipeline cycles regardless of data values.

Node 0 acts as the initiator and orchestrator. It broadcasts **x** and the MAC configuration to all remote nodes via the serial link, then triggers computation on all remote nodes before starting its own local MAC. This ordering is deliberate: since Node 0 writes its results directly into the local address without any serial link transfer, it completes faster than remote nodes. Starting remote nodes first therefore hides their communication latency behind Node 0's own computation.

In software mode, remote nodes arm `IRQ_MAC_START` (IRQ line 22) and block on `wfi` between test iterations. When Node 0 has finished broadcasting **x** and writes to `MAC_START`, the interrupt fires on all remote nodes, the ISR clears the latch by reading `MAC_START`, and sets `mac_start_received`, after which the remote node proceeds with its dot product computation.

In accelerator mode, the interrupt handler fires once per completed row (`IRQ_MAC_DONE`, line 20). Node 0 writes results directly into its local result buffer at `Y_RETURN_BASE`; remote nodes ship each row's result back to Node 0 via the serial link. Node 0's SRAM write monitor watches the `Y_RETURN_BASE` window and fires `IRQ_SRAM_MONITOR` once all `NUM_ROWS` entries have been written, at which point `y_all_received` is set and verification proceeds.

To enable a fair comparison between the software baseline and the MAC accelerator, the Croc core was configured to use the hardware multiplier by changing the following line in `rtl/core_wrap.sv`:

```
.RV32M ( cve2_pkg::RV32MNone ),
```

to:

```
.RV32M ( cve2_pkg::RV32MFast ),
```

and the Makefile compiler flag was updated from `rv32i_zicsr` to `rv32im_zicsr`. Without these changes, the software baseline would rely on a slow emulation routine (`__mulsi3`), making the comparison with the hardware-accelerated path not meaningful.

Finally, Node 0 verifies every entry of the result vector against the expected value, returning the failing row index on mismatch or 0 on success.

6 Results

As described in Section 5, three weight matrix sizes were evaluated. Since the total number of rows was not always divisible by the number of Crocs, `NUM_ROWS` was rounded up to the nearest multiple for node counts 3, 5, 6, 7, 9, 10, 11, 12, 13, 14, while 1, 2, 4, 8 required no adjustment. This rounding slightly enlarges the weight matrix for the affected configurations, and since the additional result rows must be transmitted back to the initiating Croc and the additional results need to be validated, the extra communication overhead and additional validation produce the small ripples visible in the throughput curves for these node counts.

For the smallest matrix configuration, the total row count is already so small that dividing it across many chips and rounding up to ensure equal row counts per node noticeably changes the overall weight matrix size, producing ripples in the throughput curves too large to yield a meaningful comparison.

Performance for this configuration was therefore evaluated only at 1, 2, 4, 8 nodes, where the row count divides evenly without rounding.

~512 Element $\mathbf{W}$ Matrix-Vector Multiplication without and with Accelerator

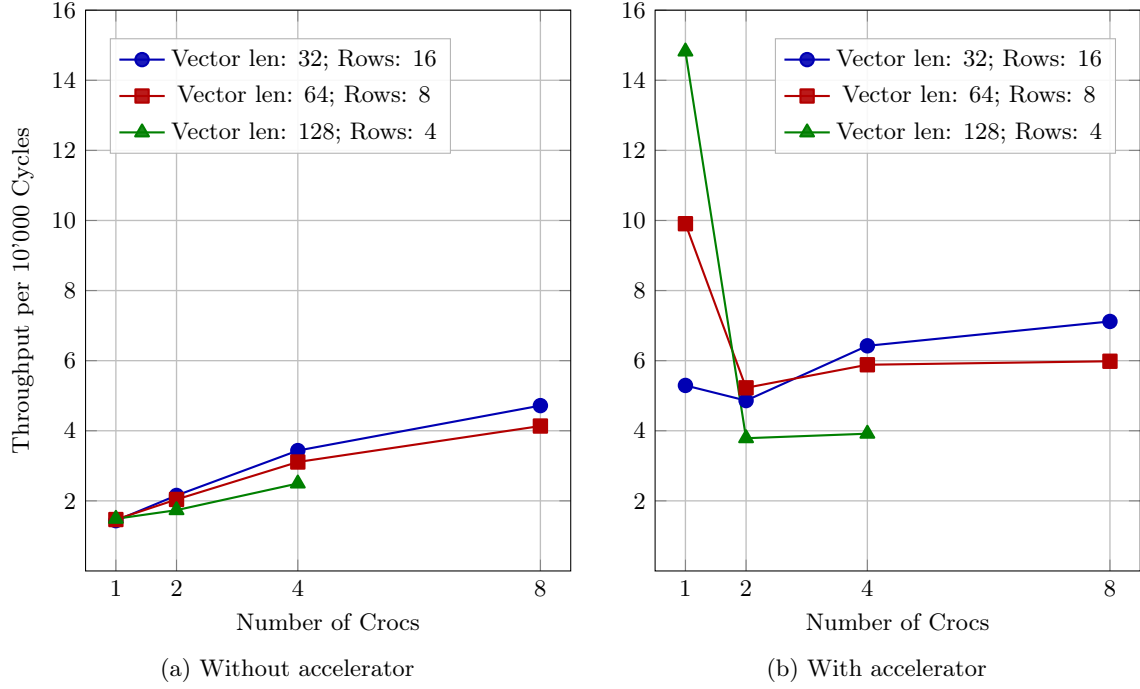


Figure 4: Throughput vs. number of Crocs for ~512 element $\mathbf{W}$ matrix-vector multiplication (matrix size ≈ 2 kB SRAM), without accelerator (left) and with accelerator (right).

~4096 Element $\mathbf{W}$ Matrix-Vector Multiplication without and with Accelerator

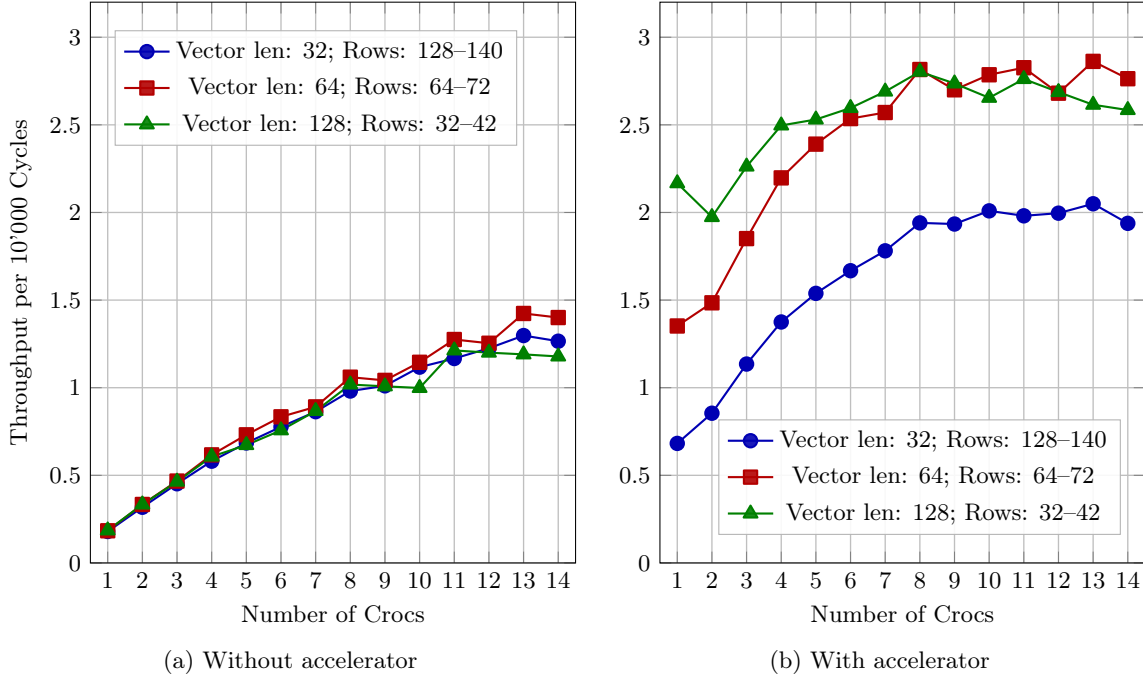


Figure 5: Throughput vs. number of Crocs for ~4096 element $\mathbf{W}$ matrix-vector multiplication (matrix size ≈ 16 kB SRAM), without accelerator (left) and with accelerator (right).

~57344 Element $\mathbf{W}$ Matrix-Vector Multiplication without and with Accelerator

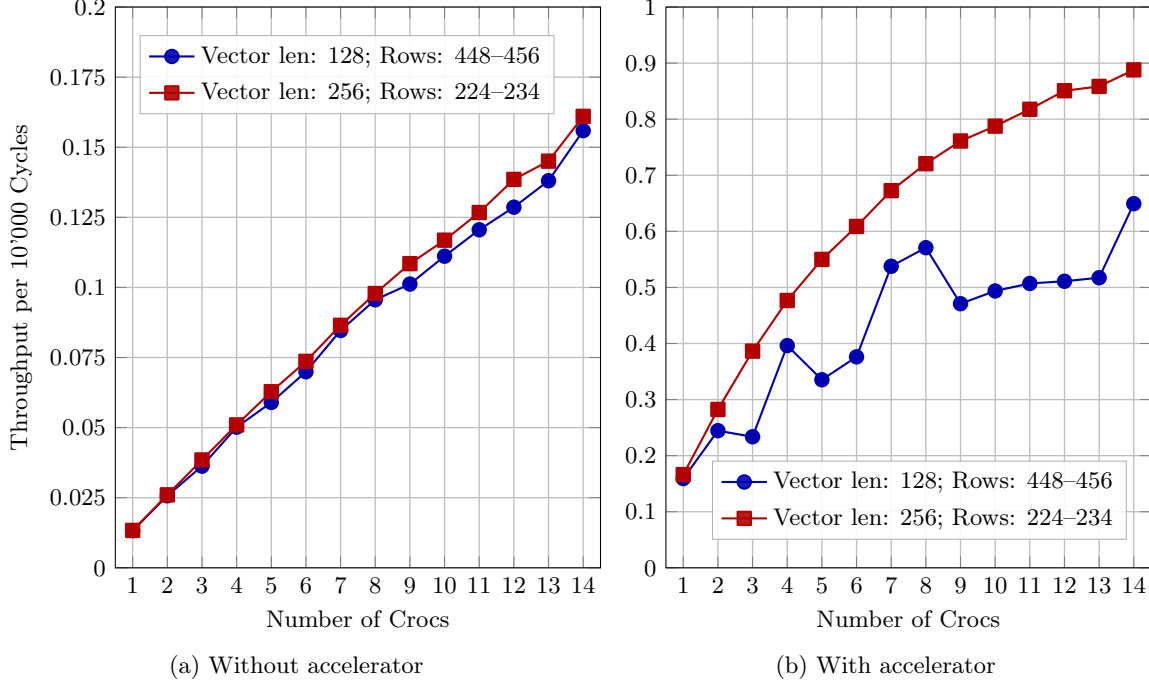


Figure 6: Throughput vs. number of Crocs for ~ 57344 element $\mathbf{W}$ matrix-vector multiplication (matrix size ≈ 224 kB SRAM), without accelerator (left) and with accelerator (right). Note that only the 14-Croc configuration represents a realistic deployment capable of being executed, as it fully utilizes the 16 kB SRAM per core to accommodate the weight matrix. The remaining configurations serve strictly as theoretical benchmarks for scaling trends. Furthermore, when comparing the two diagrams, care should be taken as their y-axes are scaled differently.

The figures show that speedup increases with matrix size. For small matrices, adding accelerated nodes off-chip actually *decreases* throughput; for all other matrix sizes, distributing computation across nodes consistently improves it.

In the non-accelerated case, the speedup from 1 to 8 nodes is modest for the ~ 512 -element $\mathbf{W}$ matrix ($\sim 2.5\times$), but substantially larger for the ~ 57344 -element matrix ($7.3\times$) over the same range.

A roofline pattern, consistent with the analytical model from Section 2.1.4, is also visible in the accelerated results: throughput saturates as the number of Croc instances grows. In Figure 5, 14 nodes achieve roughly the same speedup as 8, indicating the system has become bandwidth-bound rather than compute-bound.

The maximal end-to-end speedup - from a single, non-accelerated Croc to the best multi-Croc accelerated configuration - is matrix-size dependent:

- **Smallest matrix (~ 512 elements):** best to stay on-chip; a single accelerated Croc with a 128-element x-vector (4 rows) yields a $9.9\times$ speedup.
- **Intermediate matrix (~ 4096 elements):** the maximum speedup of $15.6\times$ is reached at 13 nodes with a 64-element x-vector, though 8 nodes achieve comparable performance due to the roofline effect.
- **Largest matrix (~ 57344 elements, only realistic at 14 nodes):** compared to a hypothetical single-Croc baseline with equivalent SRAM capacity, the theoretical speedup reaches $66.6\times$ with a 256-element x-vector - here, distributing across 14 accelerated nodes is clearly worthwhile.

6.1 Timing

The final design runs at 80 MHz with no setup or hold violations. The margin is small, however: the worst setup slack is only 0.01 ns, so the design may not reach 80 MHz in silicon.

Reaching the 80 MHz system clock required pipelining the OBI connection from the Ring Serial Link manager to the crossbar with an OBI cut module. The worst path started at the system clock, ran through the crossbar, and ended at the enable of the credit-return clock gate in the ring serial link; it was long enough that OpenROAD produced setup violations even at 70 MHz. Adding the cut introduces negligible latency on the serial link’s manager port while raising the overall chip speed considerably.

Implementing the Ring Serial Link IP correctly also required specifying the clock constraints. Transmission over the ring uses double data rate, and the fastest clock transmitted is the system clock divided by four, i.e. 20 MHz. For DDR with a clock division of four, the ring clock is phase-shifted by 90° to guarantee correct data when sampling. These constraints match those of the standard one-to-one Serial Link IP, which lets us apply them in the same way as in manufactured designs [3].

Figure 7 shows these clock constraints together with the correct timing annotations. The data travelling from node 0 to node 1 has brief intervals where it holds no valid value, caused by the switching inside the serial link module; this is harmless, since the clock constraints ensure the ring clock samples only once the data has settled. The figure also shows the credits being returned from node 1 to node 0 as a pulse at the same frequency as node 1’s system clock. Note that this simulation does not model the wire delay introduced by the connections between the manufactured chips.

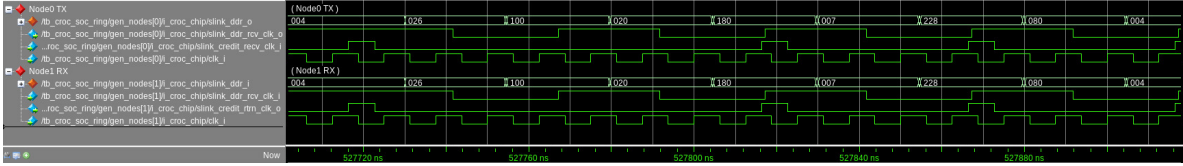


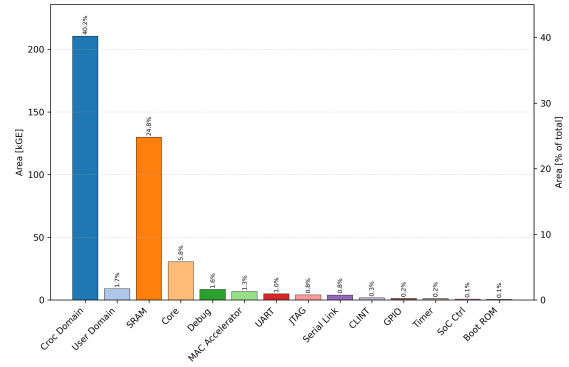
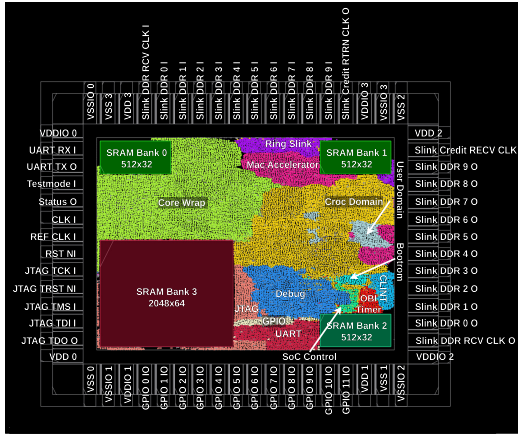
Figure 7: Post-layout simulation with Timing Annotation of Ring Serial Link connection between node 0 and node 1.

6.2 Area

As specified in the physical requirements, the final design measures $2416\mu\text{m} \times 1916\mu\text{m}$, or $2500\mu\text{m} \times 2000\mu\text{m}$ including the searing. Core utilization is 59.5 % and standard-cell utilization is 39.4 %. Density stays below 90 % throughout, but congestion reaches 100 % in some regions, which forced OpenROAD to route on the top metal layer to connect the standard cells. Despite this, the design passes DRC and LVS, with the sole exception of the metal-1 off-grid pin violations on the SRAMs, which we were instructed to ignore.

The NAND2X1 cell of the EZ130 8T library is $3.36\mu\text{m}$ high and $1.68\mu\text{m}$ wide, giving an area of $5.6448\mu\text{m}^2$. One gate equivalent therefore corresponds to $5.6448\mu\text{m}^2$, so 1 kGE equals $5644.8\mu\text{m}^2$. With a total cell area of $2953413\mu\text{m}^2$, the design amounts to 522.92 kGE.

Figures 8a and 8b show the placement and the per-module area utilization. Note that the SRAM monitor belongs to the Croc domain and the user ROM to the user domain. The Croc domain is one of the largest blocks on the chip, most likely because of the OBI crossbar, which has been substantially expanded; since its area grows quadratically with the number of ports, this expansion accounts for much of the domain’s size.



(a) OpenROAD visualization of module placement within the final design, including I/O pins.

(b) Area used by modules in percentages and kGE.

Figure 8: Module placement and per-module area utilization of the final design.

6.3 Power

Figure 9 shows that the average power consumption grows linearly as more nodes are added to the ring. With a single node, the ring draws 23.8 mW of internal power, 57.8 mW of switching power, and 7.47 μ W of leakage power, for a total of 81.6 mW.

Since the values in Figure 9 are time-averaged, this linear trend is expected. These figures are simulated, however, power estimation from simulation is never fully reliable, even less so for the ring in particular. Compared with last year's student projects [4], our design at 81.6 mW is close to the River design, which consumed 76 mW. It is worth noting, though, that the standard cells of the EZ130 8T library draw considerably less power than the SG13G2 library used last year. Taking this into account, our design is likely to fall on the higher-power end rather than the lower.

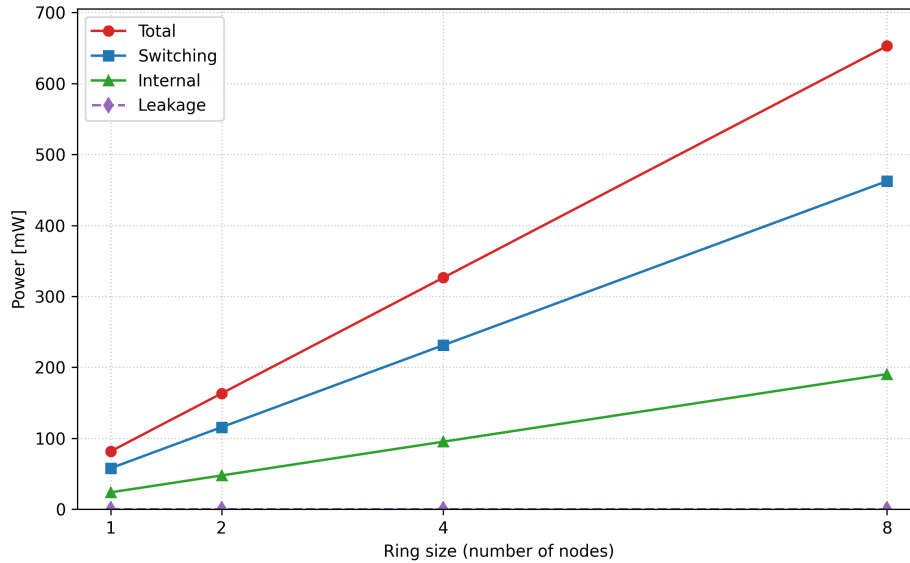


Figure 9: Average power simulation of the Ring using the MAC accelerator test program with a vector length of 64 and number of rows of 64.

7 Future Work

This project evaluated a single application of the ring bus - computationally coupling multiple Croc instances to accelerate large matrix-vector products - and demonstrated its feasibility. Other workloads with a similarly favorable compute-to-communication ratio could benefit from the same architecture, such as Monte Carlo simulations.

Additionally, since Node 0 incurs no serial-link latency for its own results, an unequal workload distribution - assigning Node 0 a larger share of rows than the remote nodes - could better balance compute and communication time. This should yield a speedup even in the small-matrix regime, where the current equal-distribution scheme is communication-bound, and is left for future exploration.

In general, the chip's functionality must be tested in silicon to confirm that it operates correctly. One concern is particularly important: the additional latency introduced by the cables in the ring could increase the time until a credit is returned so much that the ring occasionally stalls at clock division 4. This needs to be tested to ensure the ring runs at the intended speed.

References

- [1] F. Aegerter and L. M. Hausmann, “Ring serial link.” <https://github.com/faegerter/Ring-Serial-Link>, 2026. Accessed: 2026-06-17.
- [2] F. Aegerter, “Ring serial link.” https://github.com/faegerter/Ring-Serial-Link/tree/global_payloads, 2026. Accessed: 2026-06-17.
- [3] PULP Platform, “Basilisk sdc timing constraints (cheshire-ihp130-o).” <https://github.com/pulp-platform/cheshire-ihp130-o/blob/main/target/ihp13/openroad/src/basilisk.sdc>, 2025. Accessed: 2026-06-17.
- [4] Integrated Systems Laboratory, ETH Zurich, “VLSI projects 2025.” https://vlsi.ethz.ch/wiki/2025_VLSI_Projects, 2025. Accessed: 2026-06-17.